Analiza Algoritmilor – Pregătire examen 2022

1) Demonstrați că $o(g(n)) \cap \omega(g(n)) = \phi$.

Varianta 1

$o(g(n)) = \{ f(n) \mid \forall c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^* \text{ a.î. } 0 \le f(n) < c \cdot g(n), \forall n \ge n_0 \}$

$\omega(g(n)) = \{ f(n) \mid \forall c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^* \text{ a.î. } 0 \le c \cdot g(n) < f(n), \forall n \ge n_0 \}$,

Pp. prin absurd că $o(g(n)) \cap \omega(g(n)) \neq \phi \Rightarrow$

$\Rightarrow \exists h(n)$ a.î. $h(n) \in o(g(n))$ și $h(n) \in \omega(g(n))$

$h(n) \in o(g(n)) \Rightarrow \forall c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î. $0 \le h(n) < c \cdot g(n), \forall n \ge n_0$ (1)

$h(n) \in \omega(g(n)) \Rightarrow \forall c \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î. $0 \le c \cdot g(n) < h(n), \forall n \ge n_0$ (2)

$\Rightarrow$ pp. făcută este falsă.

Varianta 2

$f(n) = O(g(n)) \Rightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} \neq \infty$

$f(n) = \Omega(g(n)) \Rightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} \neq 0$

$f(n) = \Theta(g(n)) \Rightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} \neq 0$ și $\neq \infty$

$f(n) = o(g(n)) \Rightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

$f(n) = \omega(g(n)) \Rightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$

$f(n) \le c \cdot g(n)$, $\frac{f(n)}{g(n)} \le c$ $\Big\} \Rightarrow f(n) = O(g(n))$

$(A \setminus B) \cap (C \setminus B) = (A \cap C) \setminus B$, $\Theta(g(n)) = \Theta(g(n))$

$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$

$o(g(n)) = O(g(n)) \setminus \Theta(g(n))$

$\omega(g(n)) = \Omega(g(n)) \setminus \Theta(g(n))$

$$T(n) = \begin{cases} k_1, & n = 1 \\ 2T(n/2) + k_2, & n > 1 \end{cases} \qquad k_1, k_2 \in \mathbb{R}_+^*$$

Aplicăm met. substituției:

Alegem $T(n) = \Theta(n) \Rightarrow \exists c_1, c_2 \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î.

$$c_1 \cdot n \leq T(n) \leq c_2 \cdot n, \ \forall n \geq n_0$$

Caz de bază: $n = 1$

$$c_1 \leq T(1) \leq c_2 \Rightarrow \boxed{c_1 \leq k_1 \leq c_2} \qquad \boxed{n_0 = 1}$$

Pas de inducție: $n/2 \to n$

Ipoteza de inducție: $\exists c_1, c_2 \in \mathbb{R}_+^*, \exists n_0 \in \mathbb{N}^*$ a.î.

$$c_1 \cdot \frac{n}{2} \leq T(n/2) \leq c_2 \cdot \frac{n}{2}, \ \forall n \geq n_0$$

$$c_1 \cdot \frac{n}{2} \leq T(n/2) \leq c_2 \cdot \frac{n}{2}, \ \forall n \geq n_0 \ \Big| \cdot 2 + k_2$$

$$2c_1 \cdot \frac{n}{2} + k_2 \leq \underbrace{2T(n/2) + k_2}_{T(n)} \leq 2 \cdot c_2 \cdot \frac{n}{2} + k_2 \Rightarrow$$

$$\Rightarrow c_1 n + k_2 \leq T(n) \leq c_2 n + k_2$$

$$c_1 n \overset{?}{\leq} c_1 n + k_2 \leq T(n) \leq c_2 n + k_2 \overset{?}{\leq} c_2 n \Rightarrow k_2 = 0 \quad \text{Kp}$$

Putem lua: $T(n) = \Theta(2n-1)$

$$T(n) = 3T(n/7) + k_1 \cdot n \qquad \Big| \cdot 3^0$$

$$T(n/7) = 3T(n/7^2) + k_1 \cdot \frac{n}{7} \qquad \Big| \cdot 3^1$$

$$\cdots\cdots\cdots\cdots$$

$$T(n/7^k) = 3 \cdot T(n/7^{k+1}) + k_1 \cdot \frac{n}{7^k} \qquad \Big| \cdot 3^k$$

$$T(n) = 3^{k+1} \cdot T(n/7^{k+1}) + k_1 \cdot n \sum_{i=0}^{k} \frac{3^i}{7^i}$$

$$\frac{n}{7^{k+1}} = 1 \Rightarrow n = 7^{k+1} \Rightarrow k+1 = \log_7 n$$

$$T(n) = 3^{\log_7 n} \cdot T(1) + k_1 \cdot n \cdot \frac{\left(\frac{3}{7}\right)^{k+1} - 1}{\frac{3}{7} - 1}$$

$$T(n) = 3^{\log_7 n} + k_1 \cdot n \cdot \left[\left(\frac{3}{7}\right)^{\log_7 n} - 1\right] \cdot \left(\frac{7}{-4}\right) =$$

$$= 3^{\log_7 n} + k_1 \cdot n \cdot \left(1 - \frac{3^{\log_7 n}}{7^{\log_7 n}}\right) \cdot \frac{7}{4} =$$

$$= 3^{\log_7 n} + k_1 \cdot n \left(1 - \frac{3^{\log_7 n}}{n}\right) \cdot \frac{7}{4} =$$

$$= 3^{\log_7 n} + \frac{7}{4} \cdot k_1 \cdot n - \frac{7}{4} \cdot k_1 \cdot \cancel{n} \cdot \frac{3^{\log_7 n}}{\cancel{n}} =$$

$$= 3^{\log_7 n}\left(1 - \frac{7}{4} \cdot k_1\right) + \frac{7}{4} \cdot k_1 \cdot n \Rightarrow$$

$$\Rightarrow T(n) = \Theta\left(3^{\log_7 n} + n\right)$$

Care este complexitatea spațială pt. algoritmul

```
Alg (n) {
    rezultat = 0
    for i = 1.. n
        for j = 1.. n
            rezultat = rezultat + n
}
```

rezultat $\leq n^3$ → $O(\log n^3)$ → $O(3 \log n)$

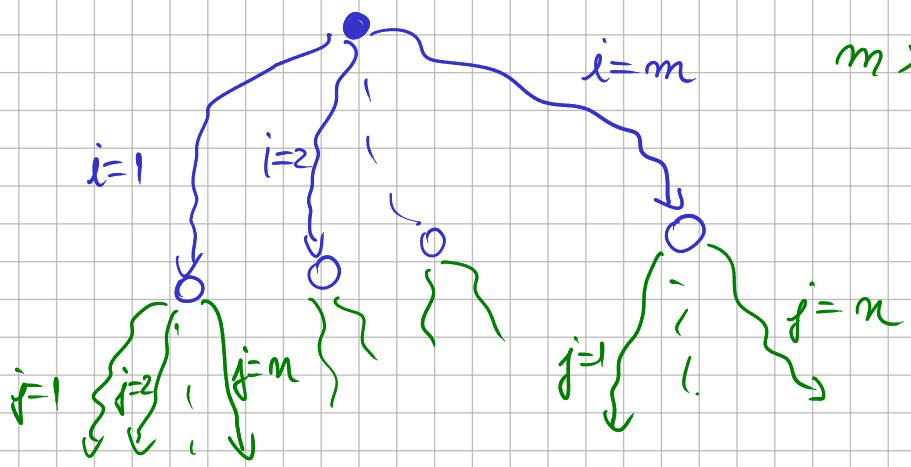
$i \leq n$ → $O(\log n)$

$j \leq n$ → $O(\log n)$

Complexitatea: $O(3 \log n + \log n + \log n) = O(\log n)$

```
NAlg (x, m, n) {
    i = choice ({1, ..., m})  --- O(1)
    j = choice ({1, ..., n})  --- O(1)

    if i + j > x
        print (i, j)          } O(1)
        succes
    fail
}
```



$m \times n$ ramuri

Tipul TList

TList := nil | cons (T, TList)

a) Constructorii de bază

nil : → TList → constructor nular (constructor extern)

cons : T × TList → TList → constructor intern

b) Operatori

head : TList \ {nil} → T

tail : TList \ {nil} → TList

null : TList → {0, 1}

c) Axiome

head (cons (x, L)) = x

tail (cons (x, L)) = L

null (nil) = 1

null (cons (x, L)) = 0

Algoritmul concat

```
concat (l1, l2)
    if l1 = nil
        return l2
    else if l1 = cons(x, l3)
        return cons(x, concat(l3, l2))
```

Dem prin inducție structurală:

$\forall l_1 \in T$ listă, $concat(l_1, nil) = l_1$

Caz de bază: aplicat pt. toți constructorii externi ai TDA-ului

$l_1 = nil$

$concat(l_1, nil) = concat(nil, nil) = nil = l_1$ ✓

Pas de inducție: aplicat pt. toți constr. interni ai TDA-ului

$L \to cons(x, L)$

Ipoteza de inducție: $concat(L, nil) = L$

$concat(cons(x, L), nil) = cons(x, L)$?!

$concat(cons(x, L), nil) = cons(x, \underbrace{concat(L, nil)}_{\text{ip. de ind.}}) =$

$= cons(x, L)$

.) Problema TSP pt. un graf $G = (V, E)$ și vrem să verificăm dacă ∃ un ciclu de cost cel mult k.

$V = \{1, 2, 3, 4\}$ → graf complet

1, 2, 3, 4, 1

1, 3, 2, 4, 1

Generez permutări de V - - - -

Generare permutări

```
∀Alg (G=(V,E), K)
    n = |V|
    perm = [0, 0, ..., 0]  // n elemente
    for i = 1..n                          ---- O(n)
        x = choice(V)                     ---- O(1)
        for j = 1..i-1                    ---- O(n)
            if perm[j] == x               } O(1)
                fail
        perm[i] = x                       ---- O(1)
    * perm conține o permutare a mulțimii V
    cost = 0
    for i = 1..n-1                        } O(n)
        cost = cost + E[perm[i]][perm[i+1]]
    cost = cost + E[perm[n]][perm[1]]     } O(1)
    if cost <= k
        succes
    fail
```

⊛ Cum pot verifica dacă este K costul celui mai mic ciclu?

Fie o relație R peste o mulțime X a.î. orice șir de forma

$\ldots\ldots R x_n R \ldots R x_2 R x_1$ cu elemente din X este finit $\Rightarrow$

$\Rightarrow$ R este o relație bine formată

Exemplu: $<$ pt. $\mathbb{N}$

$0<1<2<3<4<5$